

markpublish Benutzerhandbuch

Moderne PDF-Dokumentenerstellung aus Markdown mit Typst

Praxisorientierte Anleitung und vollständige Referenz zur Konfiguration, CLI-Bedienung, Theme-Gestaltung und Dokumentenerstellung mit markpublish.

Version	2.0.0
Datum	04.09.2026
Autor	Frank Winter
Copyright	© 2026 Frank Winter
Status	Freigegeben

Inhaltsverzeichnis

1 Einleitung	4
1.1 Motivation und Zielsetzung	5
1.2 Kernfeatures	5
1.3 Architektur im Überblick	5
2 Installation und Schnellstart	7
2.1 Voraussetzungen	8
2.2 Installation	8
2.3 Schritte bis zum ersten Build	8
3 Kommandozeilen-Schnittstelle (CLI)	10
3.1 Befehlsübersicht	11
3.2 markpublish build	11
3.3 markpublish init	12
3.4 markpublish manual / markpublish cheatsheet	12
3.5 markpublish templates	13
3.6 markpublish export-template	14
3.7 markpublish labels	15
3.8 Globale Optionen	16
4 Projektkonfiguration mit markpublish.yaml	17
4.1 Die drei Strukturebenen	18
4.2 Beispiel einer vollständigen Konfiguration	19
5 Templates und Mehrsprachigkeit	22
5.1 Die Auflösungshierarchie für Themes	23
5.2 Aufbau eines Themes	23
5.3 Eigene Themes erstellen und anpassen	26
5.4 Mehrsprachigkeit und Textkaskade (i18n)	26
6 Besondere Markdown-Features	28
6.1 Hinweisboxen (Admonitions und Callouts)	29
6.2 Definitionslisten	30
6.3 Aufgabenlisten (Tasklists)	31
6.4 Mathematische Formeln	32
6.5 Alltägliche Sonderzeichen	32

Anhänge

Anhang A: Schema-Referenz markpublish.yaml	34
A.1 Dokument-Ebene (document)	34
A.2 Globale Projekt-Einstellungen	35
A.3 Abschnitts-Ebene (parts)	35
A.4 Kapitel-Ebene (chapters)	36

A.5 Verzeichnis-Steuerung (Scope-Werte)	38
Anhang B: Fehlerbehebung und FAQ	39
B.1 Diagnose von Kompilierungsfehlern	39
B.2 Häufige Ursachen und Lösungen	39
B.3 Häufig gestellte Fragen (FAQ)	40

KAPITEL 1

Einleitung

Motivation, Kernfeatures und die Architektur von markpublish.

AUF EINEN BLICK

1.1 Motivation und Zielsetzung	5
1.2 Kernfeatures	5
1.3 Architektur im Überblick	5

1.1 Motivation und Zielsetzung

Markdown ist das führende Format für technische Dokumentationen, Notizen und Projektberichte. Seine Stärke liegt in der Einfachheit und Lesbarkeit im reinen Textformat. Sobald jedoch aus diesen Texten hochwertige, druckfertige Berichte, Handbücher oder Whitepapers entstehen sollen, stoßen herkömmliche Werkzeuge an ihre Grenzen. Häufig folgt ein zeitaufwendiger manueller Nachbearbeitungsschritt in Textverarbeitungs- oder Layoutprogrammen.

markpublish schließt diese Lücke: Es verwandelt strukturierte Markdown-Dateien über eine einfache, aber mächtige Konfiguration vollautomatisch in typografisch anspruchsvolle, professionell gestaltete PDF-Dokumente.

Das Ziel von markpublish ist es, Entwicklern, technischen Redakteuren und Autoren einen nahtlosen Veröffentlichungsprozess zu bieten. Inhalte werden in einfachem Markdown verfasst, während markpublish die vollständige Kontrolle über Layout, Gliederung, Trennseiten, Nummerierung, Verzeichnisse und Typografie übernimmt.

1.2 Kernfeatures

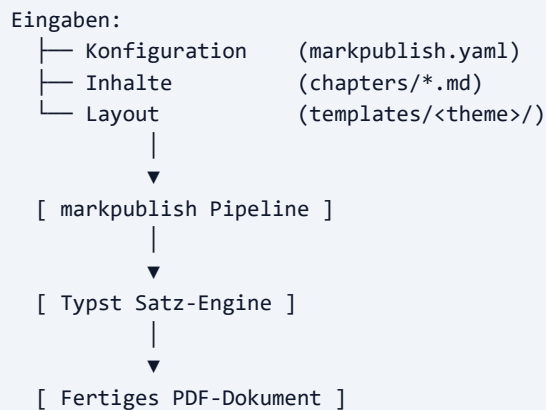
markpublish zeichnet sich durch einen klaren Fokus auf Dokumentenqualität, Geschwindigkeit und Verlässlichkeit aus:

- **Moderne Satzqualität durch Typst:** Durch die Verwendung der innovativen Satz-Engine Typst entstehen Dokumente mit herausragender typografischer Präzision, perfektem Randausgleich und ästhetischem Layout.
- **Extrem hohe Kompiliergeschwindigkeit:** Selbst umfangreiche Dokumente mit dutzenden Seiten, Abbildungen und Tabellen werden in rund ein bis zwei Sekunden vollständig gesetzt.
- **Einfache Installation und Portabilität:** Als reguläres Python-Paket lässt sich markpublish plattformübergreifend auf Windows, macOS und Linux ohne komplizierte Einrichtung betreiben.
- **Deklarative Konfiguration:** Eine einzige Datei (`markpublish.yaml`) steuert das gesamte Projekt.
- **Mehrstufige Dokumentenarchitektur:** Unterstützung einer klaren Zweiteilung in übergeordnete Abschnitte (*Parts*) und inhaltstragende Kapitel (*Chapters*) mit flexibler Trennseiten-Steuerung.
- **Präzise Verzeichnisse und Nummerierung:** Vollautomatische Generierung von Gesamtinhaltsverzeichnis, Abschnittsverzeichnissen und lokalen Kapitelverzeichnissen sowie flexibel konfigurierbare Kapitel- und Überschriftennummerierung.
- **Umfassende Markdown-Erweiterungen:** Standardmäßige Unterstützung von GitHub-konformen Hinweisboxen (Admonitions/Callouts), Tabellen, Definitionenlisten, Fußnoten, Aufgabenlisten und Quellcode mit Syntax-Highlighting.
- **Kaskadierende Mehrsprachigkeit (i18n):** Integrierte Unterstützung für mehrsprachige Templates, um statische Texte (wie Autor, Inhaltsverzeichnis, Seite) in der richtigen Sprache anzuzeigen¹

1.3 Architektur im Überblick

¹Aktuell wird Deutsch und Englisch unterstützt.

Die Arbeitsweise von markpublish folgt einer klaren Verarbeitungs-Pipeline:



1. **Konfigurations- und Dokumentenanalyse:** markpublish liest die `markpublish.yaml` ein, validiert sämtliche Einstellungen und baut die Gliederungshierarchie aus Abschnitten und Kapiteln auf.
2. **Inhaltsaufbereitung:** Die Markdown-Dateien der Kapitel werden eingelesen und strukturiert aufbereitet. Überschriften erhalten ihre exakte Nummerierung, Sprungmarken werden gesetzt und Verzeichniseinträge synchron erfasst.
3. **Template-Zusammenstellung:** Das gewählte Theme (standardmäßig das integrierte Standard-Theme) stellt die Layout-Vorgaben bereit. Metadaten, konfigurierte Kopf- und Fußzeilen, Trennseiten und Textinhalte werden präzise in die Typst-Umgebung übergeben.
4. **Kompilierung zum PDF:** Die Typst-Engine kompiliert das Gesamtdokument in einem einzigen, hocheffizienten Durchlauf direkt in die fertige PDF-Datei.

KAPITEL 2

Installation und Schnellstart

Der direkte Weg von der Installation bis zum ersten fertigen PDF.

AUF EINEN BLICK

2.1 Voraussetzungen	8
2.2 Installation	8
2.3 Schritte bis zum ersten Build	8

Dieses Kapitel beschreibt den direkten Weg von der Installation bis zum ersten fertig erstellten PDF-Dokument.

2.1 Voraussetzungen

Für den Betrieb von markpublish wird eine funktionierende Python-Installation benötigt:

- **Python-Version:** 3.10 oder neuer
- **Betriebssystem:** Windows, macOS oder Linux

2.2 Installation

markpublish wird komfortabel über den Python-Paketmanager `pip` installiert:

```
pip install markpublish
```

Nach Abschluss der Installation steht der Befehl `markpublish` (sowie das kurze Alias `mpub`) im Terminal zur Verfügung. Die erfolgreiche Bereitstellung lässt sich mit folgendem Aufruf prüfen:

```
markpublish --version
```

2.3 Schritte bis zum ersten Build

Die Erstellung eines neuen Dokumentenprojekts erfolgt in wenigen einfachen Schritten:

2.3.1 Neues Projekt initialisieren

Mit dem Befehl `init` legt markpublish ein neues Projektverzeichnis mit einer minimalen, lauffähigen Startstruktur an:

```
markpublish init mein-dokument
```

Dieser Befehl erzeugt einen neuen Ordner `mein-dokument/` mit folgender Grundstruktur:

```
mein-dokument/  
├─ markpublish.yaml  
└─ next-steps.md
```

2.3.2 In das Projektverzeichnis wechseln

Wechseln Sie in das neu angelegte Verzeichnis:

```
cd mein-dokument
```


2.3.3 Erstes PDF erstellen

Starten Sie den Veröffentlichungsprozess direkt aus dem Projektverzeichnis heraus:

```
markpublish build
```

markpublish liest die Konfigurationsdatei `markpublish.yaml`, verarbeitet die referenzierten Markdown-Dateien und kompiliert das Dokument.

2.3.4 Dokument öffnen

Das generierte PDF befindet sich nun direkt in Ihrem Projektverzeichnis:

```
mein-dokument/  
└─ new_document.pdf
```

(Hinweis: Der Dateiname der Ausgabe leitet sich automatisch aus dem in der `markpublish.yaml` hinterlegten Dokumententitel ab; standardmäßig ist dies `New Document` → `new_document.pdf`.)

Öffnen Sie die Datei mit einem beliebigen PDF-Betrachter, um das Ergebnis zu begutachten. Das Dokument enthält die formatierte Einstiegsseite mit Kopf- und Fußzeilen. Sobald Ihr Dokument wächst, können Sie in der `markpublish.yaml` einfach `cover: true` und `document_toc: "full"` (oder eine Zahl für die Verzeichnistiefe) aktivieren, um ein gestaltetes Deckblatt und ein Inhaltsverzeichnis einzublenden.

KAPITEL 3

Kommandozeilen-Schnittstelle (CLI)

Detaillierte Befehlsübersicht und vollständige Optionen-Referenz aller Befehle.

AUF EINEN BLICK

3.1 Befehlsübersicht	11
3.2 markpublish build	11
3.3 markpublish init	12
3.4 markpublish manual / markpublish cheatsheet	12
3.5 markpublish templates	13
3.6 markpublish export-template	14
3.7 markpublish labels	15
3.8 Globale Optionen	16

Die Kommandozeile ist die primäre Schnittstelle zur Arbeit mit markpublish. Neben dem Hauptbefehl `markpublish` steht das kurze Alias `mpub` zur Verfügung.

3.1 Befehlsübersicht

Befehl	Kurzbeschreibung
<code>build</code>	Kompiliert das Dokument auf Basis der Konfigurationsdatei.
<code>init</code>	Erstellt ein neues Projekt mit Konfigurationsdatei und Beispielkapitel.
<code>manual</code>	Rendert das offizielle Benutzerhandbuch aus dem installierten Paket.
<code>cheatsheet</code>	Rendert die kompakte zweiseitige Kurzreferenz aus dem Paket.
<code>templates</code>	Listet alle verfügbaren Templates und deren Herkunftsebenen auf.
<code>export-template</code>	Exportiert das Standard-Theme zur individuellen Anpassung in das Projekt.
<code>labels</code>	Zeigt alle aufgelösten Textbeschriftungen und deren Herkunftsebene (i18n) an.

3.2 markpublish build

Der Befehl `build` führt den Veröffentlichungsprozess aus. Er liest die Konfiguration ein, verarbeitet alle Markdown-Dateien der Kapitel und kompiliert das fertige Dokument.

```
markpublish build [CONFIG_FILE] [OPTIONEN]
```

3.2.1 Optionen

Option	Default	Bedeutung
<code>CONFIG_FILE</code> (<i>Argument</i>)	<code>markpublish.yaml</code>	Pfad zur Projekt-Konfigurationsdatei.
<code>--target, -t</code>	<code>pdf</code>	Ausgabeformat (pdf).
<code>--output, -o</code>	(<i>automatisch</i>)	Pfad zur Zielfeile oder zum Zielverzeichnis. Endet der Pfad ohne Dateieindung, wird er als Verzeichnis behandelt.
<code>--templates-dir</code>	<code>None</code>	Pfad zu einem benutzerdefinierten Vorlagen-Verzeichnis.

3.2.2 Erläuterungen

Wird kein `--output` angegeben, erzeugt markpublish die Datei im selben Verzeichnis wie die Konfigurationsdatei (standardmäßig im Projektordner). Der Dateiname leitet sich automatisch aus dem Dokumententitel ab (z. B. `mein_dokument.pdf`).

Beispiele:

```
# Standard-Build im aktuellen Projekt
markpublish build

# Anderes Konfigurationsfile und Ausgabeordner festlegen
markpublish build projekte/bericht.yaml --output dist/
```

3.3 markpublish init

Der Befehl `init` dient dem schnellen Aufsetzen eines neuen Projekts. Er erzeugt das Verzeichnis (sofern noch nicht vorhanden), eine lauffähige `markpublish.yaml` sowie ein erstes Markdown-Kapitel.

```
markpublish init [ZIELORDNER] [OPTIONEN]
```

3.3.1 Optionen

Option	Default	Bedeutung
<code>ZIELORDNER</code> (<i>Argument</i>)	<code>.</code> (<i>aktueller Ordner</i>)	Zielverzeichnis, in dem das Projekt initialisiert werden soll.
<code>--title, -t</code>	New Document	Titel des neuen Dokuments in der erzeugten Konfiguration.
<code>--lang, -l</code>	(<i>Systemsprache</i>)	Sprache der Vorlage (de, en).

3.3.2 Erläuterungen

Die erzeugte `markpublish.yaml` und das Einstiegskapitel stammen aus den mitgelieferten Vorlagen. Mit `--lang` (z. B. `--lang de`) lässt sich die Sprache der Vorlage explizit vorgeben; ohne Angabe wird automatisch die Systemsprache ermittelt und eingetragen, sodass das Projekt auf jedem Rechner konsistent kompiliert.

Beispiel:

```
markpublish init mein-handbuch --lang de --title "Benutzerhandbuch System X"
```

3.4 markpublish manual / markpublish cheatsheet

markpublish bringt seine vollständige Dokumentation direkt im Paket mit. Beide Befehle erzeugen die Dokumentation on-demand aus den installierten Quellen – das Ergebnis beschreibt somit immer exakt die aktuell installierte Version.

- `markpublish manual`: Rendert dieses umfassende Benutzerhandbuch.
- `markpublish cheatsheet`: Rendert eine kompakte Schnellreferenz.

```
markpublish manual [OPTIONEN]
markpublish cheatsheet [OPTIONEN]
```

3.4.1 Optionen

Option	Default	Bedeutung
<code>--lang, -l</code>	<i>(Systemsprache)</i>	Sprachauswahl des Dokuments (z. B. <code>de</code> oder <code>en</code>).
<code>--target, -t</code>	<code>pdf</code>	Ausgabeformat (<code>pdf</code>).
<code>--output, -o</code>	<code>.</code> <i>(aktueller Ordner)</i>	Zielpfad für das generierte Dokument.
<code>--theme</code>	<code>None</code>	Render-Vorgabe mit einem alternativen Theme anstelle des Standard-Themes.
<code>--templates-dir</code>	<code>None</code>	Pfad zu einem benutzerdefinierten Vorlagen-Verzeichnis.

3.4.2 Erläuterungen

Wird beim Aufruf von `manual` oder `cheatsheet` keine Sprache mit `--lang` angegeben, ermittelt markpublish automatisch die Sprache Ihres Betriebssystems. Ist für diese Sprache keine Übersetzung vorhanden, wird auf die erste im Paket vorhandene Sprache zurückgegriffen (beim Handbuch derzeit `de`).

Für eigene Dokumente gilt: Die Sprachwahl (`language: "de"`) in der `markpublish.yaml` steuert die statischen Textvariablen (wie „Inhaltsverzeichnis“, „Kapitel“, „Seite X von Y“). Diese müssen vom verwendeten Theme in dessen `i18n.yaml` bereitgestellt bzw. unterstützt werden.

Beispiel:

```
# Deutsches Benutzerhandbuch im aktuellen Ordner erzeugen
markpublish manual --lang de

# Englische Kurzreferenz erzeugen
markpublish cheatsheet --lang en
```

3.5 markpublish templates

Listet alle im System und Projekt verfügbaren Vorlagen auf und zeigt übersichtlich an, welche Vorlage im Falle von Namensgleichheiten Vorrang hat.

```
markpublish templates [OPTIONEN]
```

3.5.1 Optionen

Option	Default	Bedeutung
<code>--target, -t</code>	None	Filtert die Anzeige nach Zielformat (<code>pdf</code>).
<code>--templates-dir</code>	None	Zusätzliches Vorlagen-Verzeichnis, das in die Suche einbezogen werden soll.

3.5.2 Erläuterungen

Die Ausgabe erfolgt als tabellarische Übersicht mit Angabe von Theme-Name, Herkunftsebene (User, Common oder Package) und einem Status-Indikator, welches Theme bei einem Build aktiv verwendet wird.

3.6 markpublish export-template

Exportiert ein mitgeliefertes Theme aus dem markpublish-Paket in ein lokales Verzeichnis, damit es nach eigenen Wünschen angepasst oder als Vorlage für ein eigenes Corporate Design verwendet werden kann.

```
markpublish export-template [THEME] [ZIELVERZEICHNIS] [OPTIONEN]
```

3.6.1 Optionen

Option	Default	Bedeutung
<code>THEME</code> (<i>Argument</i>)	<code>default</code>	Name des zu exportierenden Themes.
<code>ZIELVERZEICHNIS</code> (<i>Argument</i>)	<code>templates</code>	Ordner, in den die Vorlagendateien kopiert werden sollen.
<code>--target, -t</code>	<code>all</code>	Zielformat der Vorlagen (<code>pdf</code> oder <code>all</code>).

3.6.2 Erläuterungen

Beim Export werden die Typst-Vorlagendateien (`template.typ`) sowie die zugehörige Beschriftungsdatei `i18n.yaml` in das Zielverzeichnis kopiert. Liegt ein Theme im Projektordner unter `templates/`, greift markpublish automatisch darauf zu.

Beispiel:

```
# Exportiert das Standard-Theme in den lokalen Ordner ./templates
markpublish export-template default ./templates
```

3.7 markpublish labels

Zeigt eine detaillierte Aufstellung aller statischen Beschriftungen (wie „Inhaltsverzeichnis“, „Kapitel“, „Seite X von Y“) für die im Projekt gewählte Sprache sowie deren Herkunftsebene in der Kaskade.

```
markpublish labels [CONFIG_FILE] [OPTIONEN]
```

3.7.1 Optionen

Option	Default	Bedeutung
CONFIG_FILE (<i>Argument</i>)	markpublish.yaml	Pfad zur Projekt-Konfigurationsdatei.
--target, -t	pdf	Zielformat, dessen Beschriftungskaskade angezeigt werden soll.
--templates-dir	None	Benutzerdefiniertes Vorlagen-Verzeichnis.
--overridden	False	Zeigt ausschließlich Beschriftungen an, die durch ein Theme oder Projekt überschrieben wurden.

3.7.2 Erläuterungen

Dieser Befehl ist besonders hilfreich bei der Erstellung eigener Themes oder neuer Übersetzungen, um zu prüfen, ob alle benötigten Textbausteine vollständig vorhanden sind.

Die Spalte **i18n-Quelle** nennt die Ebene der Kaskade, aus der ein Text stammt:

Wert	Datei
mpub	Der Programmstandard, markpublish/i18n.yaml
theme	<theme>/i18n.yaml
target	<theme>/<zielformat>/i18n.yaml
projekt	i18n.yaml neben Ihrer markpublish.yaml

Grün hervorgehoben sind die Ebenen, die den Programmstandard ersetzt haben — genau diese zeigt `--overridden` allein. Der Sprachblock steht nur dann in Klammern dabei, wenn er von der

Dokumentsprache abweicht: (*) für einen sprachunabhängigen Eintrag, (en) für einen Rückfall auf die Fallback-Sprache. Am Fuß der Ausgabe stehen die vollständigen Pfade zu allen vier Ebenen.

Zusätzlich prüft der Befehl das Theme gegen den Aufruf, den markpublish beim Rendern erzeugt. Ein Theme, dessen `setup-document` einen gesendeten Parameter nicht deklariert, wird hier gemeldet — vor dem Bauen statt währenddessen.

3.8 Globale Optionen

Folgende Optionen stehen global für alle Befehle zur Verfügung:

Option	Bedeutung
<code>--version, -v</code>	Gibt die installierte Version von markpublish aus und beendet das Programm.
<code>--help</code>	Zeigt die integrierte Hilfe und Parameterübersicht im Terminal an.

Projektkonfiguration mit markpublish.yaml

Strukturierung in Document, Parts und Chapters anhand praktischer Beispiele.

AUF EINEN BLICK

4.1 Die drei Strukturebenen	18
4.2 Beispiel einer vollständigen Konfiguration	19

Die zentrale Steuerungsdatei eines jeden markpublish-Projekts ist die Datei `markpublish.yaml`. Sie definiert das Dokument deklarativ: Metadaten, Layout-Vorgaben, Gliederung und die Zuordnung der Markdown-Dateien zu Kapiteln und Abschnitten.

4.1 Die drei Strukturebenen

markpublish organisiert Dokumente in drei klar voneinander abgegrenzten Ebenen:

```
document
├── parts (Abschnitte)
│   └── chapters (Kapitel)
│       └── [Unterkapitel in chapters]
```

4.1.1 Document (Dokumentebene)

Metadaten Titel, Untertitel, Autoren, Version, Datum, Sprache, Status und Copyright-Hinweise.

Layout-Schalter Anzeige des Deckblatts (`cover`), Aktivierung von Kopfzeilen (`header`) und Fußzeilen (`footer`).

Globale Verzeichnisvorgaben Steuerung der Tiefe für das Gesamt-Inhaltsverzeichnis (`document_toc`), Abschnittsverzeichnisse (`part_toc`) und lokale Kapitelverzeichnisse (`chapter_toc`).

Nummerierungsregeln Vorgaben zur automatischen Nummerierung von Überschriften (`autnum_style`, `autnum_from_level`, `autnum_prefix`).

4.1.1.1 Eigene Metadatenfelder

Unter `document:` sind über die bekannten Schlüssel hinaus **beliebige eigene Felder** erlaubt. Sie erscheinen ohne weiteres Zutun im Metadatenraster des Deckblatts:

```
document:
  title: "Projektbericht"
  abteilung: "Controlling"
  kunde: "Musterunternehmen GmbH"
  freigegeben: true
```

Drei Dinge sind dabei zu wissen:

- **Die Beschriftung kommt aus der i18n-Kaskade.** Findet sich dort kein Eintrag für den Schlüssel, druckt das Deckblatt den Schlüssel selbst — also `abteilung` statt `Abteilung`. Legen Sie dafür eine `i18n.yaml` neben Ihre `markpublish.yaml`:

```
# i18n.yaml, im Projektverzeichnis
de:
  abteilung: "Abteilung"
  kunde: "Kunde"
  freigegeben: "Freigegeben"
```

Diese Datei ist die letzte Stufe der Kaskade und gewinnt damit gegen Programm und Theme (siehe Kapitel *Template- und Designsystem*).

- **Der Typ bleibt erhalten.** Ein `true` ist ein Wahrheitswert und kein Text: das Deckblatt setzt `ja` beziehungsweise `yes`, je nach Dokumentsprache. Die beiden Wörter stehen als `bool_true` und `bool_false` in der i18n-Kaskade.
- **Ein Tippfehler wird gedruckt, nicht gemeldet.** `title:` statt `title:` ergibt kein Fehlerbild, sondern eine zusätzliche Zeile auf dem Deckblatt. `markpublish labels` zeigt jedes Feld mit seinem Befund an — ein Blick dorthin vor dem ersten Bauen erspart die Suche.

4.1.2 Parts (Abschnitte)

Ein Dokument wird durch `parts:` in übergeordnete Abschnitte gegliedert (z. B. „Hauptteil“, „Fallstudien“, „Anhänge“):

- Ein Abschnitt fasst logisch zusammengehörige Kapitel zusammen.
- Abschnitte können mit Trennseiten (`break_before: "divider"`) oder einfachen Seitenumbrüchen (`break_before: "page"`) eingeleitet werden.
- Auf Abschnittsebene gesetzte Einstellungen (z. B. Verzeichnistiefe oder Nummerierungspräfixe wie `autonum_prefix: "A."`) vererben sich automatisch auf alle darin enthaltenen Kapitel.

4.1.3 Chapters (Kapitel)

Die Liste `chapters:` innerhalb eines Abschnitts verweist auf die eigentlichen Markdown-Inhaltsdateien:

- Jedes Kapitel verweist auf seine Markdown-Datei (`file:`). Autoren können in ihren Markdown-Dateien ganz natürlich mit einer #-Überschrift beginnen.
- Über `show_title:` (`true` oder `false`, Standard: `true`) lässt sich steuern, ob die #-Überschrift der Markdown-Datei auf der Inhaltsseite gerendert werden soll. Wird vor dem Kapitel eine Trennseite erzeugt (`break_before: "divider"`), kann mit `show_title: false` verhindert werden, dass der Titel auf der Folgeseite doppelt erscheint — die Inhaltsseite beginnt dann direkt mit dem Fließtext oder der ersten Zwischenüberschrift.
- Über die optionalen Schlüssel `toc_title:` (für Inhaltsverzeichnisse und Kopfzeilen) und `divider_title:` (für Trennseiten) können gezielt abweichende Titel vergeben werden (z. B. eine prägnante Kurzform im Inhaltsverzeichnis gegenüber einer ausführlichen Überschrift auf der Textseite). Fehlen die Schlüssel, erben beide automatisch die #-Überschrift der Datei.
- Soll eine Trennseite erzeugt werden (`break_before: "divider"`) oder das Kapitel im Inhaltsverzeichnis gelistet werden, muss mindestens eine #-Überschrift oder der entsprechende Titelschlüssel (`divider_title` / `toc_title`) vorhanden sein, andernfalls bricht der Build mit einer klaren Fehlermeldung ab.
- Kapitel können über das Feld `break_before` steuern, ob vor ihnen eine Trennseite erzeugt wird, ein einfacher Seitenwechsel erfolgt oder der Text nahtlos anschließt.
- Kapitel können untereinander angeordnet werden; die inhaltliche Nummerierungs- und Gliederungstiefe (1.1, 1.1.1) entsteht dabei ausschließlich aus den Überschriftenebenen (H2, H3) innerhalb der Markdown-Dateien.

4.2 Beispiel einer vollständigen Konfiguration

Das folgende Beispiel zeigt eine praxisnahe, vollständige `markpublish.yaml`:

```
# markpublish.yaml

document:
  title: "Projektbericht Jahresabschluss"
  subtitle: "Analyse, Ergebnisse und Ausblick"
  summary: "Umfassender Gesamtbericht über die Projektphasen des vergangenen
Geschäftsjahres."
  author: "Projektgruppe Controlling"
  date: "auto"
  version: "1.2.0"
  language: "de"
  copyright: "© 2026 Musterunternehmen GmbH"

# Layout-Elemente
cover: true
header: true
footer: true

# Verzeichnistiefen
document_toc: 2
part_toc: 2
chapter_toc: "none"

# Nummerierung
autonum_style: "decimal"
autonum_from_level: 1

theme: "default"

parts:
  - part: "Hauptteil"
    break_before: "none"
    chapters:
      - file: "chapters/01_einleitung.md"
        break_before: "page"

      - file: "chapters/02_analyse.md"
        # Kurzform für das Inhaltsverzeichnis bei langer Datei-H1:
        toc_title: "Marktanalyse"
        break_before: "page"

      - file: "chapters/03_ergebnisse.md"
        break_before: "divider"

  - part: "Anhänge"
    break_before: "divider"
    autonum_from_level: 2
    autonum_prefix: "A."
    chapters:
      - file: "chapters/anhang_tabellen.md"

      - file: "chapters/anhang_glossar.md"
```

Eine vollständige Übersicht aller verfügbaren Schlüssel, Datentypen, Standardwerte und Kombinationsmöglichkeiten für jede der drei Ebenen finden Sie in **Anhang A: Schema-Referenz markpublish.yaml** am Ende dieses Handbuchs.

Templates und Mehrsprachigkeit

Theme-Struktur mit Typst, 3-stufige Auflösungshierarchie und Textkaskade (i18n).

AUF EINEN BLICK

5.1 Die Auflösungshierarchie für Themes	23
5.2 Aufbau eines Themes	23
5.3 Eigene Themes erstellen und anpassen	26
5.4 Mehrsprachigkeit und Textkaskade (i18n)	26

markpublish trennt Inhalt und Gestaltung strikt voneinander: Die Textinhalte entstehen in Markdown, während das visuelle Erscheinungsbild, die Typografie und das Seitenlayout über Typst-Templates gesteuert werden. Ergänzt wird dieses System durch eine flexible Kaskade zur Mehrsprachigkeit (i18n).

5.1 Die Auflösungshierarchie für Themes

Wenn markpublish nach einem Theme sucht (angegeben über `theme`: in der Konfigurationsdatei oder den Standard `default`), durchsucht der Resolver folgende Ebenen der Reihe nach (der erste Treffer gewinnt):

1. Benutzer-Vorlagen (user):

Vorlagen im Home-Verzeichnis des aktuellen Benutzers (`~/.markpublish/templates/<theme>/`) bzw. im benutzerspezifischen AppData-Verzeichnis. Dies ermöglicht autorenweite Standardvorlagen über alle Projekte hinweg.

2. Projekt- / Vorlagen-Verzeichnis (common):

Vorlagen im Projektordner (`./templates/<theme>/`) oder in einem explizit über `templates_dir`: in der `markpublish.yaml`, über das Kommandozeilen-Flag `--templates-dir` oder die Umgebungsvariable `MARKPUBLISH_TEMPLATES_DIR` festgelegten Verzeichnis.

3. Paket-Vorlagen (package):

Das mitgelieferte Standard-Theme `default`. Es dient als verlässlicher Fallback, wenn auf Benutzer- oder Projektebene keine Vorlage gefunden wird.

5.2 Aufbau eines Themes

Ein vollständiges markpublish-Theme besitzt folgende Verzeichnisstruktur:

```
templates/  
└─ mein-theme/  
   └─ i18n.yaml           # Übergreifende Beschriftungen des Themes  
      └─ pdf/  
         └─ template.typ  # Typst-Layoutvorlage für das PDF  
         └─ i18n.yaml     # Formatspezifische Beschriftungen des Themes  
         └─ fonts/       # Optionale Schriftdateien (z. B. .ttf, .otf)  
         └─ assets/      # Statische Grafiken, Logos und Icons  
            └─ icons/
```

5.2.1 Typst als Layout-Engine

Typst ist eine moderne, hochperformante Satz- und Programmiersprache für Dokumente. Sie bietet mächtige Funktionen für Seitenränder, Kopf- und Fußzeilen, Farbwelten, Schriftdefinitionen und mathematischen Formelsatz.

• Weiterführende Typst-Dokumentation:

Eine vollständige Einführung in alle Möglichkeiten von Typst (Gestaltungselemente, Funktionen, Regeln und Typografie) finden Sie in der offiziellen Dokumentation unter <https://typst.app/docs>.

5.2.2 Exemplarischer Aufbau einer `template.typ`

Die Datei `template.typ` definiert das Gesamterscheinungsbild des Dokuments. Ein Auszug zeigt die grundlegende Struktur unter Verwendung moderner Typst-Kontexte (`context`):

```
// template.typ (Auszug)

#let setup-document(
  language: "de",
  show-cover: true,
  show-toc: true,
  toc-title: "Inhaltsverzeichnis",
  meta: (:),
  labels: (:),
  body
) = {
  let title = meta.at("title").value
  // Grundlegende Seiteneigenschaften
  set page(
    paper: "a4",
    margin: (top: 2.5cm, bottom: 2.5cm, left: 2.5cm, right: 2.5cm),
    header: context {
      // Individuelle Kopfzeilengestaltung
      text(9pt, fill: rgb("#64748b"))[#title]
    },
    footer: context {
      // Seitennummerierung über aktuelle Labels
      let page_num = counter(page).display()
      let page_label = labels.at("page", default: "Seite")
      align(center)[#text(9pt)[#page_label #page_num]]
    }
  )

  // Grundschriftart und Absatzgestaltung
  set text(font: "Open Sans", size: 10pt, lang: "de")
  set par(justify: true, leading: 0.65em)

  body
}
```

5.2.3 Das `meta`-Wörterbuch

Sämtliche Dokumentangaben erreichen `setup-document` über das Wörterbuch `meta` — Titel und Version genauso wie ein frei ergänztes `abteilung`. Eigene Parameter dafür gibt es nicht: zwei Wege zur selben Angabe können auseinanderlaufen. Jeder Eintrag ist ein Datensatz mit vier Feldern:

```
meta.at("author")
// -> (key: "author", label: "Autor", value: "Frank Winter", in-grid: true)
```

Feld	Bedeutung
key	Der Schlüssel aus der <code>markpublish.yaml</code>

Feld	Bedeutung
label	Die Beschriftung aus der i18n-Kaskade, oder <code>none</code>
value	Der Wert — mit seinem Typ : Text, Zahl, Wahrheitswert oder Liste
in-grid	<code>false</code> für Titel, Untertitel und Summary; sie stehen oben auf dem Deckblatt und gehören nicht noch einmal ins Metadatenraster

Der Vorteil: Das Theme muss nicht wissen, welche Felder es gibt. Es zählt auf, was da ist — und erreicht damit auch die eigenen Felder, die ein Dokument unter `document:` ergänzt:

```
for item in meta.values().filter(it => it.in-grid and it.value != none) {  
  [#if item.label != none { item.label } else { item.key }]: #item.value  
}
```

5.2.4 Reihenfolge auf dem Titelblatt

Welche Angabe im Metadatenraster zuerst steht, entscheidet das Theme — eine Zeile in `template.typ`:

```
#let cover-order = ("version", "date", "author", "copyright", "status")
```

Schlüssel, die dort nicht vorkommen — Ihre eigenen Felder aus der `markpublish.yaml` —, folgen dahinter in der Reihenfolge der Konfiguration. Ändern Sie die Zeile, ändert sich das Titelblatt; markpublish reicht die Angaben nur weiter und mischt sich nicht ein.

Wichtig

`meta.at("kunde")` **ohne** `default:` bricht den Build ab, wenn das Dokument den Schlüssel nicht kennt. markpublish meldet das vorher mit Fundstelle und Abhilfe; markpublish `labels` zeigt es ebenfalls an. Wer eine Angabe optional halten will, notiert einen Fallback: `meta.at("kunde", default: (value: ""))`.

Weil der Typ erhalten bleibt, entscheidet das Theme über die Schreibweise. Ein Wahrheitswert wird über die Kaskade formuliert statt als `true` gedruckt:

```
#let meta-value(value, labels) = {  
  if type(value) == bool {  
    if value { labels.at("bool_true", default: "Ja") } else { labels.at("bool_false",  
default: "Nein") }  
  } else if type(value) == array {  
    value.map(v => str(v)).join(", ")  
  } else {  
    str(value)  
  }  
}
```

5.3 Eigene Themes erstellen und anpassen

Der einfachste und sicherste Weg zur Erstellung eines eigenen Corporate Designs ist der Export des integrierten Standard-Themes:

```
markpublish export-template default ./templates --target pdf
```

Dieser Befehl kopiert das Standard-Theme vollständig in das lokale Verzeichnis `templates/default/`. Sie können die Dateien anschließend direkt bearbeiten, Schriften anpassen, Farben ändern oder Ihr Firmenlogo einbinden. markpublish greift beim nächsten `build` automatisch auf Ihre angepasste Projektvorlage zu.

5.4 Mehrsprachigkeit und Textkaskade (i18n)

Professionelle Dokumente enthalten eine Vielzahl statischer Texte, die nicht aus den Markdown-Kapiteln stammen, sondern vom Template erzeugt werden – beispielsweise „Inhaltsverzeichnis“, „Kapitel“, „Seite X von Y“ oder Hinweise im Deckblatt.

markpublish verwaltet diese Texte über ein Kaskadensystem in `i18n.yaml`-Dateien.

5.4.1 Die 4-stufige Beschriftungskaskade

Bei der Auflösung eines Textschlüssels (z. B. `toc_title`) sucht markpublish in folgender Reihenfolge – spätere Fundstellen überschreiben frühere:

1. **Paket-Basis** (`markpublish/i18n.yaml`): Vollständige Standardbeschriftungen für alle unterstützten Sprachen.
2. **Theme-Ebene** (`<theme>/i18n.yaml`): Themes können eigene Formulierungen oder Bezeichnungen definieren.
3. **Format-Ebene** (`<theme>/pdf/i18n.yaml`): Formatspezifische Anpassungen des Themes.
4. **Projekt-Ebene** (`i18n.yaml` **neben der** `markpublish.yaml`): Texte dieses einen Dokuments — mit höchster Priorität.

Die Projekt-Ebene ist der Ort für die Beschriftung eigener Metadatenfelder: Wer unter `document:` ein `abteilung:` "F&E" notiert, schreibt hier `abteilung:` "Abteilung" dazu. Ohne diesen Eintrag druckt das Deckblatt den Schlüssel selbst. Sie können damit auch einen Text des Themes für ein einzelnes Dokument ersetzen, ohne das Theme zu kopieren.

5.4.2 Aufbau der `i18n.yaml`

Eine `i18n.yaml` enthält strukturierte Übersetzungen je Sprachkürzel:

```
de:
  toc_title: "Inhaltsverzeichnis"
  chapter: "Kapitel"
  part: "Abschnitt"
  page: "Seite"
```

```
page_of: "von"
version: "Version"
author: "Autor"

en:
  toc_title: "Table of Contents"
  chapter: "Chapter"
  part: "Part"
  page: "Page"
  page_of: "of"
  version: "Version"
  author: "Author"
```

5.4.3 Spracherkennung und Validierung

Die Sprache eines Dokuments wird in der `markpublish.yaml` über den Schlüssel `language:` (z. B. `language: "de"`) festgelegt. Fehlt dieser Eintrag, ermittelt markpublish automatisch die Systemsprache des Autors.

Mit dem CLI-Befehl `markpublish labels` können Sie jederzeit prüfen, welche Beschriftungen für Ihr Projekt aktiv sind und aus welcher Datei sie stammen.

Besondere Markdown-Features

Auszeichnung und typografische Darstellung von Hinweisboxen, Definitionslisten, Aufgaben und mathematischen Formeln.

AUF EINEN BLICK

6.1 Hinweisboxen (Admonitions und Callouts)	29
6.2 Definitionslisten	30
6.3 Aufgabenlisten (Tasklists)	31
6.4 Mathematische Formeln	32
6.5 Alltägliche Sonderzeichen	32

markpublish unterstützt den vollen Funktionsumfang des modernen CommonMark- und GitHub Flavored Markdown-Standards. Grundlegende Elemente wie Überschriften, Absätze, Textauszeichnungen (*kursiv*, **fett**), Tabellen, Aufzählungen, Quellcode-Blöcke und Weblinks funktionieren exakt wie gewohnt.

Dieses Kapitel konzentriert sich ausschließlich auf die darüber hinausgehenden Spezialfeatures und typografischen Erweiterungen von markpublish.

6.1 Hinweisboxen (Admonitions und Callouts)

Zur optischen Hervorhebung wichtiger Passagen unterstützt markpublish GitHub-konforme Call-out-Blöcke.

6.1.1 Syntax

```
> [!NOTE]
> Dies ist ein allgemeiner Informationstext mit nützlichen Details.

> [!TIP]
> Ein praktischer Tipp für schnellere Arbeitsabläufe.

> [!IMPORTANT]
> Wichtige Information, die für das Verständnis unerlässlich ist.

> [!WARNING]
> Warnung vor möglichen Fehlbedienungen oder unerwartetem Verhalten.

> [!CAUTION]
> Kritischer Sicherheitshinweis vor potenziellen Datenverlusten.
```

6.1.2 Live-Darstellung im Dokument

Die obige Auszeichnung wird im PDF wie folgt als gestaltete Boxen mit farbigem Rahmen und Icon gerendert:

Hinweis

Dies ist ein allgemeiner Informationstext mit nützlichen Details.

Tipp

Ein praktischer Tipp für schnellere Arbeitsabläufe.

Wichtig

Wichtige Information, die für das Verständnis unerlässlich ist.

Warnung

Warnung vor möglichen Fehlbedienungen oder unerwartetem Verhalten.

Achtung

Kritischer Sicherheitshinweis vor potenziellen Datenverlusten.

6.1.3 Automatische Lokalisierung über i18n-Labels

Die Titelleiste der Hinweisboxen („Hinweis“, „Tipp“, „Wichtig“, „Warnung“, „Achtung“) wird automatisch über die Beschriftungsdatei `i18n.yaml` der gewählten Dokumentensprache bezogen:

Box-Typ	Verwendeter i18n-Schlüssel	Deutsche Beschriftung
<code>[!NOTE]</code>	<code>alert_note</code>	Hinweis
<code>[!TIP]</code>	<code>alert_tip</code>	Tipp
<code>[!IMPORTANT]</code>	<code>alert_important</code>	Wichtig
<code>[!WARNING]</code>	<code>alert_warning</code>	Warnung
<code>[!CAUTION]</code>	<code>alert_caution</code>	Achtung

Wird im Markdown ein eigener Titel gewünscht, kann dieser in der ersten Zeile direkt angegeben werden:

```
> [!NOTE] Individuelle Überschrift  
> Eigener Inhalt mit spezifischem Titel.
```

Individuelle Überschrift

Eigener Inhalt mit spezifischem Titel.

6.2 Definitionslisten

Für Glossare, Begriffserklärungen oder Parameterübersichten bieten Definitionslisten eine besonders saubere typografische Struktur.

6.2.1 Syntax

Markdown
: Einfache, lesbare Auszeichnungssprache für formatierte Texte im Klartextformat.

markpublish.yaml
: Zentrale Projekt-Konfigurationsdatei zur Steuerung von Dokumentstruktur, Metadaten und Nummerierung.

Theme
: Gestaltungsvorlage aus Typst-Templates und Beschriftungen, die das visuelle Layout des Dokuments bestimmt.

Typst
: Moderne, hochperformante Satz-Engine, die markpublish zur typografischen Erzeugung von Druck-PDFs nutzt.

6.2.2 Live-Darstellung im Dokument

Markdown Einfache, lesbare Auszeichnungssprache für formatierte Texte im Klartextformat.
markpublish.yaml Zentrale Projekt-Konfigurationsdatei zur Steuerung von Dokumentstruktur, Metadaten und Nummerierung.
Theme Gestaltungsvorlage aus Typst-Templates und Beschriftungen, die das visuelle Layout des Dokuments bestimmt.
Typst Moderne, hochperformante Satz-Engine, die markpublish zur typografischen Erzeugung von Druck-PDFs nutzt.

6.3 Aufgabenlisten (Tasklists)

Zur Darstellung von Checklisten, To-Do-Listen oder Meilensteinen stehen Aufgabenlisten zur Verfügung. markpublish setzt Aufgabenlisten typografisch sauber ohne vorangestellte Aufzählungspunkte (Bullets) direkt mit der Checkbox und dem Text; mehrzeilige Beschreibungen brechen bündig unter der ersten Zeile um.

6.3.1 Syntax

- [x] Python 3.10 oder neuer bereitstellen
- [x] markpublish installieren
- [] Erstes eigenes Dokument veröffentlichen

6.3.2 Live-Darstellung im Dokument

☒ Python 3.10 oder neuer bereitstellen

- ☒ markpublish installieren
- ☐ Erstes eigenes Dokument veröffentlichen

6.4 Mathematische Formeln

markpublish ermöglicht das direkte Setzen mathematischer Formeln im Fließtext oder als abgesetzte Formelblöcke. Die Formeln werden über die Satz-Engine Typst typografisch präzise gerendert.

Unterstützt werden gängige mathematische Ausdrücke wie Brüche (`\frac{a}{b}`), Wurzeln (`\sqrt{x}`), Summen (`\sum`), Integrale (`\int`), griechische Symbole (`\alpha`, `\pi`, `\sigma` etc.) sowie die native Typst-Math-Syntax.

6.4.1 Syntax

Die berühmte Energie-Masse-Äquivalenz lautet $E = mc^2$.

Abgesetzte Kreisflächenberechnung und quadratische Lösungsformel:

$$A = \pi \cdot r^2 \quad \text{und} \quad x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Typografischer Hinweis zu Variablen:

In Typst Math stehen mehrbuchstabige Wörter für Funktionen oder Bezeichner (wie `sin`, `cos`, `sqrt`). Die Multiplikation einzelner Variablen wird deshalb durch ein Leerzeichen getrennt notiert (z. B. `m c^2` oder `4 a c`).

6.4.2 Live-Darstellung im Dokument

Die berühmte Energie-Masse-Äquivalenz lautet $E = mc^2$.

Abgesetzte Kreisflächenberechnung und quadratische Lösungsformel:

$$A = \pi \cdot r^2 \quad \text{und} \quad x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

6.5 Alltägliche Sonderzeichen

Viele Dokumentationswerkzeuge erfordern das umständliche Maskieren bestimmter Symbole. markpublish verarbeitet gebräuchliche Zeichen im Fließtext vollautomatisch und kollisionsfrei:

- **Währungsbeträge:** `$5.00` oder `10.50 $` werden nicht versehentlich als mathematische Formeln interpretiert.
- **Programmiersprachen:** Namen wie `c#` lösen keine Typst-Sonderbefehle aus.
- **Benutzernamen und Erwähnungen:** `@author` oder `user@domain.com` werden sicher gedruckt.
- **Spitze Klammern:** Technische Platzhalter wie `<zielverzeichnis>` bleiben im Text erhalten.

Anhänge

Vollständige Spezifikations-Tabellen und Hilfestellungen zur Fehlerbehebung.

INHALT DIESES ABSCHNITTS

Anhang A: Schema-Referenz markpublish.yaml	34
A.1 Dokument-Ebene (document)	34
A.2 Globale Projekt-Einstellungen	35
A.3 Abschnitts-Ebene (parts)	35
A.4 Kapitel-Ebene (chapters)	36
A.5 Verzeichnis-Steuerung (Scope-Werte)	38
Anhang B: Fehlerbehebung und FAQ	39
B.1 Diagnose von Kompilierungsfehlern	39
B.2 Häufige Ursachen und Lösungen	39
B.3 Häufig gestellte Fragen (FAQ)	40

Anhang A: Schema-Referenz markpublish.yaml

Dieser Anhang enthält die vollständige Spezifikation aller Konfigurationsoptionen der markpublish.yaml.

A.1 Dokument-Ebene (document)

Die Sektion document: legt globale Metadaten, Layoutschalter, Verzeichnisvorgaben und Nummerierungsstile für das Gesamtdokument fest.

Schlüssel	Typ	Default	Beschreibung
title	String	(Pflichtfeld)	Titel des Dokuments (erscheint auf Deckblatt, Kopfzeilen und Metadaten).
subtitle	String	None	Untertitel des Dokuments.
summary	String	None	Zusammenfassung oder Abstract (erscheint auf dem Deckblatt).
author	String	None	Name des Autors oder der Organisation.
status	String	None	Status des Dokuments (z. B. „Entwurf“, „Freigegeben“, „Vertraulich“).
copyright	String	None	Copyright-Hinweis (z. B. „© 2026 Frank Winter“).
date	String	"auto"	Datum des Dokuments. Bei "auto" oder "today" wird das aktuelle Tagesdatum eingesetzt.
version	String	None	Versionskennung (z. B. "2.0.0"). Wird nur gedruckt, wenn explizit gesetzt.
language	String	(Systemsprache)	ISO-Sprachcode (z. B. "de" oder "en"). Steuert Silbentrennung und UI-Texte.
cover	Boolean	true	Steuert, ob eine gestaltete Deckblattseite erzeugt wird.
header	Boolean	true	Aktiviert oder deaktiviert die laufende Kopfzeile im Dokument.
footer	Boolean	true	Aktiviert oder deaktiviert die laufende Fußzeile (inkl. Seitennummerierung).

Schlüssel	Typ	Default	Beschreibung
<code>document_toc</code>	Scope	<code>"full"</code>	Tiefe des Haupt-Inhaltsverzeichnisses (<code>"none"</code> , <code>"full"</code> oder Zahl ≥ 1).
<code>part_toc</code>	Scope	<code>"full"</code>	Standardtiefe für lokale Inhaltsverzeichnisse auf Abschnitts-Trennseiten.
<code>chapter_toc</code>	Scope	<code>"full"</code>	Standardtiefe für lokale Inhaltsverzeichnisse auf Kapitel-Trennseiten.
<code>autonum_style</code>	String	<code>"decimal"</code>	Nummerierungsstil: <code>"decimal"</code> (1.2.3), <code>"legal"</code> , <code>"roman"</code> oder <code>"none"</code> .
<code>autonum_from_level</code>	Integer	<code>1</code>	Überschriftenebene, ab der nummeriert wird (1 = ab H1, 2 = erst ab H2).
<code>autonum_prefix</code>	String	<code>None</code>	Zeichenkette vor den Nummern (z. B. <code>"A."</code> für Anhänge).
<code>autonum_reset</code>	Boolean	<code>false</code>	Setzt den Überschriftenzähler bei jedem neuen Kapitel auf 1 zurück.
<code>pagenum_reset</code>	Boolean	<code>false</code>	Startet die Seitennummerierung bei jedem Abschnitt oder Kapitel neu bei Seite 1.

A.2 Globale Projekt-Einstellungen

Direkt auf oberster Ebene der `markpublish.yaml` (neben `document:` und `parts:`) können folgende Schalter gesetzt werden:

Schlüssel	Typ	Default	Beschreibung
<code>theme</code>	String	<code>"default"</code>	Name des zu verwendenden Themes.
<code>templates_dir</code>	String	<code>None</code>	Optional er benutzerdefinierter Pfad zu einem Verzeichnis mit Themes.

A.3 Abschnitts-Ebene (parts)

Die Liste `parts:` unterteilt das Dokument in übergeordnete Abschnitte. Ein Abschnitt fasst Kapitel zusammen und vererbt seine Einstellungen nach unten.

Schlüssel	Typ	Default	Beschreibung
part	String	<i>(Pflichtfeld)</i>	Name des Abschnitts (z. B. „Hauptteil“, „Anhänge“).
toc_title	String	None	Optionaler abweichender Titel für das Inhaltsverzeichnis und Kopfzeilen (Standard: part).
divider_title	String	None	Optionaler abweichender Titel auf der Abschnitts-Trennseite (Standard: part).
subtitle	String	None	Untertitel des Abschnitts (erscheint auf der Trennseite).
summary	String	None	Kurzbeschreibung des Abschnitts (erscheint auf der Trennseite).
break_before	String	"divider"	Umbruchverhalten vor dem Abschnitt: "divider" (Trennseite), "page" (Seitenwechsel) oder "none".
document_toc	Scope	None	Überschreibt den Beitrag dieses Abschnitts zum Haupt-Inhaltsverzeichnis.
part_toc	Scope	None	Steuert das lokale Inhaltsverzeichnis auf der Abschnitts-Trennseite.
chapter_toc	Scope	None	Vererbt die Vorgabe für Kapitelverzeichnisse an alle Kapitel des Abschnitts.
autonum_style	String	None	Überschreibt den Nummerierungsstil für alle Kapitel des Abschnitts.
autonum_from_level	Integer	None	Überschreibt die Startebene der Nummerierung für den Abschnitt.
autonum_prefix	String	None	Präfix für Nummern innerhalb des Abschnitts (z. B. "A. ").
autonum_reset	Boolean	None	Steuert, ob Kapitel im Abschnitt jeweils bei 1 neu nummerieren.
pagenum_reset	Boolean	None	Setzt den Seitenzähler zu Beginn des Abschnitts auf 1 zurück.
chapters	Liste	<i>(Pflichtfeld)</i>	Liste der Inhaltskapitel innerhalb dieses Abschnitts (mindestens 1 Eintrag).

A.4 Kapitel-Ebene (chapters)

Die Liste `chapters`: definiert die Inhaltsdateien. Kapitel können unter `parts`: oder verschachtelt innerhalb anderer Kapitel liegen.

Die Überschrift auf der Inhaltsseite wird standardmäßig durch die führende #-Überschrift der Markdown-Datei bestimmt (kann über `show_title: false` unterdrückt werden). Für Verzeichnisse und Trennseiten stehen zwei explizite, optionale Schlüssel bereit:

Schlüssel	Typ	Default	Beschreibung
<code>file</code>	String	<code>None</code>	Relativer Pfad zur Markdown-Datei (z. B. <code>"chapters/01_intro.md"</code>).
<code>show_title</code>	Boolean	<code>true</code>	Steuert, ob die #-Überschrift der Datei auf der Inhaltsseite gedruckt wird.
<code>toc_title</code>	String	<code>None</code>	Titel für das Inhaltsverzeichnis (Haupt- und Part-TOC) sowie Kopfzeilen. Standard: Datei-H1.
<code>divider_title</code>	String	<code>None</code>	Titel auf der Kapitel-Trennseite (<code>break_before: "divider"</code>). Standard: Datei-H1.
<code>subtitle</code>	String	<code>None</code>	Untertitel des Kapitels (erscheint auf Trennseiten).
<code>summary</code>	String	<code>None</code>	Kurzbeschreibung (wird auf Trennseiten genutzt).
<code>break_before</code>	String	<code>"page"</code>	Umbruch vor dem Kapitel: <code>"page"</code> (neue Seite), <code>"divider"</code> (Trennseite) oder <code>"none"</code> .
<code>document_toc</code>	Scope	<code>None</code>	Beitrag dieses Kapitels zum Hauptverzeichnis (<code>"none"</code> , <code>"full"</code> , Zahl).
<code>chapter_toc</code>	Scope	<code>None</code>	Lokales Verzeichnis auf der Trennseite dieses Kapitels.
<code>autonum_style</code>	String	<code>None</code>	Nummerierungsstil für Überschriften dieses Kapitels.
<code>autonum_from_level</code>	Integer	<code>None</code>	Startebene der Nummerierung innerhalb des Kapitels.
<code>autonum_prefix</code>	String	<code>None</code>	Präfix für Überschriftennummern dieses Kapitels.
<code>autonum_reset</code>	Boolean	<code>None</code>	Setzt den Nummerierungszähler zu Beginn dieses Kapitels auf 1 zurück.
<code>pagenum_reset</code>	Boolean	<code>None</code>	Setzt die Seitennummerierung zu Beginn dieses Kapitels auf 1 zurück.
<code>chapters</code>	Liste	<code>[]</code>	Optionale Liste von weiteren Unterkapiteln.

Wichtig

Auf `parts:` und `chapters:` sind **nur** die hier aufgeführten Schlüssel erlaubt. Ein unbekannter Schlüssel bricht den Build ab und nennt, falls vorhanden, den ähnlich geschriebenen — `break_befor` also, bevor das Kapitel still mit dem falschen Umbruch gesetzt wird. Freie Felder gibt es ausschließlich unter `document:`; dort erreichen sie das Theme, hier bleiben sie wirkungslos.

A.5 Verzeichnis-Steuerung (Scope-Werte)

Die Verzeichnisschalter `document_toc`, `part_toc` und `chapter_toc` akzeptieren folgende Werte:

Wert	Bedeutung
"none"	Schaltet das jeweilige Verzeichnis komplett ab bzw. nimmt das Element nicht auf.
"full"	Nimmt alle vorhandenen Überschriftenebenen ohne Tiefenbegrenzung auf.
Ganze Zahl (z. B. 1, 2, 3)	Begrenzt die Verzeichnistiefe exakt auf die angegebene Zahl von Ebenen.

Anhang B: Fehlerbehebung und FAQ

Dieser Anhang bietet konkrete Hilfestellungen bei typischen Problemen während des Veröffentlichungsprozesses sowie Antworten auf häufige Fragen.

B.1 Diagnose von Kompilierungsfehlern

Sollte ein Kompilervorgang mit einer Fehlermeldung der Typst-Engine abbrechen, speichert markpublish den vollständig generierten Typst-Quellcode automatisch im aktuellen Arbeitsverzeichnis:

```
.markpublish/last_failed_build.typ
```

B.1.1 Vorgehen zur Fehleranalyse

1. Öffnen Sie die Datei `.markpublish/last_failed_build.typ` in einem beliebigen Texteditor.
2. Suchen Sie nach dem in der Fehlermeldung genannten Begriff, Variablennamen oder Textfragment.
3. Anhand des umgebenden Kontexts lässt sich unmittelbar erkennen, welches Markdown-Kapitel oder welcher Formatierungsblock die Ursache war.

B.2 Häufige Ursachen und Lösungen

B.2.1 Bilder oder Grafiken werden nicht gefunden

Problem: Typst bricht mit einer Meldung wie `image not found` ab.

Lösung:

- Pfade zu Abbildungen im Markdown (z. B. `![Diagramm](images/architektur.png)`) werden immer **relativ zur jeweiligen Markdown-Datei** aufgelöst.
- Liegt die Datei `01_kapitel.md` im Ordner `chapters/`, muss der Ordner `images/` entweder in `chapters/` oder `images/` liegen oder als `../images/architektur.png` referenziert werden.
- markpublish kopiert lokale Bilddateien während des Builds automatisch in den internen Build-Ordner.

B.2.2 Ungültige Einrückungen in der Konfiguration (YAML)

Problem: markpublish meldet beim Start `Configuration error: mapping values are not allowed here` oder `YAML parsing error`.

Lösung:

- YAML reagiert strikt auf falsche Einrückungen. Verwenden Sie für Einrückungen stets **zwei Leerzeichen** und niemals Tabulatoren.
- Achten Sie darauf, dass Listeneinträge (-) und verschachtelte Schlüssel bündig zueinander stehen.

B.2.3 Anpassung statischer Textbeschriftungen (i18n)

Problem: Ein generierter Text (wie „Inhaltsverzeichnis“ oder „Kapitel“) soll umformuliert werden oder fehlt in einer neuen Sprache.

Lösung:

- Führen Sie `markpublish labels` aus, um alle aufgelösten Textvariablen und deren Herkunftsebene anzuzeigen.
- Ergänzen oder überschreiben Sie die gewünschten Schlüssel entweder direkt in einer projektlokalen `i18n.yaml` (im Ordner neben der `markpublish.yaml`) oder in der `i18n.yaml` Ihres Themes.

B.2.4 Schriften und Schriftfamilien

Problem: Fehlermeldungen bezüglich Schriftarten oder unerwartetes Schriftbild.

Lösung:

- markpublish liefert die hochwertige serifenlose Schriftfamilie **Open Sans** direkt im Paket mit.
- Das Standard-Theme greift automatisch auf diese Schrift zu. Sie müssen auf Ihrem Betriebssystem keine Schriften manuell nachinstallieren.
- Wenn Sie in eigenen Themes andere Schriften nutzen möchten, können Sie die Schriftdateien (`.ttf`, `.otf`) direkt im Ordner `pdf/fonts/` Ihres Themes ablegen oder auf dem System installieren. markpublish bindet Theme-Schriften automatisch in den Suchpfad ein.

B.3 Häufig gestellte Fragen (FAQ)

B.3.1 Wie verhindere ich eine Trennseite vor einem Kapitel?

Standardmäßig leitet ein Kapitel mit `break_before: "page"` ein, während Abschnitte (`parts:`) mit `break_before: "divider"` eine repräsentative Trennseite erzeugen. Wenn ein Kapitel oder Abschnitt direkt ohne neue Trennseite anschließen soll, setzen Sie:

```
break_before: "none"
```

B.3.2 Wie kann ich die Seitennummerierung für jedes Kapitel neu starten?

Setzen Sie in der `markpublish.yaml` auf Dokument- oder Kapitel-Ebene:

```
pagenum_reset: true
```


markpublish setzt die Seitenzahl zu Beginn des Kapitels automatisch auf 1 zurück und berechnet die Gesamtzahl der Seiten konsistent.

B.3.3 Werden Hyperlinks im PDF klickbar exportiert?

Ja. Sowohl interne Verweise (aus dem Inhaltsverzeichnis oder Fußnoten) als auch externe Weblinks (`[Webseite](https://example.com)`) werden als echte, anklickbare PDF-Hyperlinks erzeugt.